

Technical Architecture

Overview

The **DarshanEase** application follows a modern three-tier architecture consisting of the **Presentation Layer**, **Application Layer**, and **Data Layer**. This architecture ensures scalability, maintainability, security, and efficient communication between the user interface and the database.

1. Presentation Layer (Frontend)

The Presentation Layer is responsible for interacting with users and displaying information. It provides a responsive and intuitive interface for devotees and administrators.

Technologies Used

- React.js
- TypeScript
- Tailwind CSS
- HTML5
- CSS3

Responsibilities

- User registration and login
- Display temple information
- Show available darshan slots
- Book and manage tickets
- Display booking history
- Admin dashboard for managing temples and bookings
- Form validation and user interaction

2. Application Layer (Backend)

The Application Layer contains the business logic and handles communication between the frontend and the database through REST APIs.

Technologies Used

- Node.js
- Express.js

- JWT Authentication
- RESTful APIs

Responsibilities

- User authentication and authorization
- Booking validation
- Slot availability management
- Temple management
- Booking confirmation and cancellation
- Error handling
- API request processing
- Data validation

3. Data Layer (Database)

The Data Layer stores all application data securely and ensures data consistency.

Technology Used

- MongoDB

Collections

- Users
- Temples
- Darshan Slots
- Bookings
- Admins

Responsibilities

- Store user information
- Maintain temple details
- Store booking records
- Track available time slots
- Retrieve booking history

- Backup and data integrity

Authentication Architecture

The application uses **JSON Web Token (JWT)**-based authentication.

User Authentication Flow

1. User registers using email and password.
2. Password is securely encrypted before storage.
3. User logs in with valid credentials.
4. Backend verifies the credentials.
5. JWT token is generated.
6. Token is sent to the frontend.
7. Frontend includes the token in API requests.
8. Backend validates the token before granting access to protected resources.

API Architecture

The frontend communicates with the backend using RESTful APIs over HTTP/HTTPS.

Example API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	User login
GET	/api/temples	Retrieve all temples
GET	/api/slots/:templeId	View available darshan slots
POST	/api/bookings	Book a darshan ticket
GET	/api/bookings/user	Retrieve user booking history
DELETE	/api/bookings/:id	Cancel a booking
GET	/api/admin/bookings	View all bookings (Admin)

Data Flow

1. User accesses the DarshanEase application.
2. Frontend sends API requests to the backend.
3. Backend validates the request.
4. Business logic processes the request.
5. MongoDB stores or retrieves data.
6. Backend returns the response.
7. Frontend displays updated information to the user.

Security Measures

- JWT-based authentication
- Password encryption using hashing algorithms
- Input validation
- Protected API routes
- Role-based access control (Admin/User)
- Secure database communication
- Error and exception handling

Deployment Architecture

Client

- Web Browser



Frontend

- React.js Application

↓ REST API

Backend

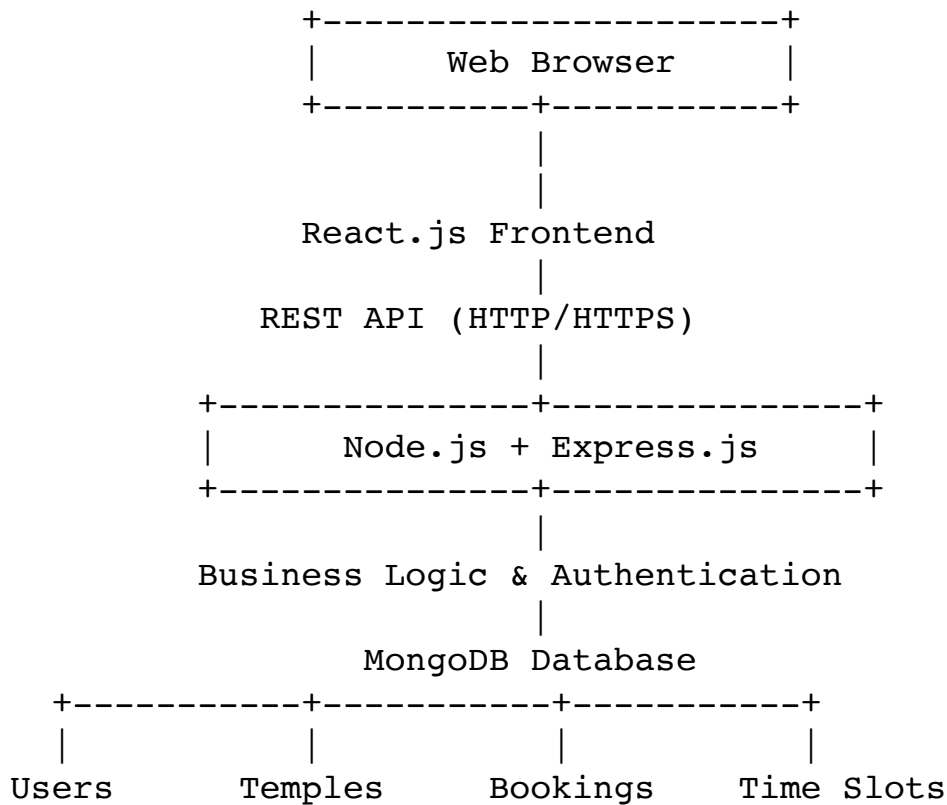
- Node.js + Express.js Server



Database

- MongoDB

Technical Architecture Diagram



Advantages of the Architecture

- Modular and scalable design
- Separation of concerns
- Easy maintenance and updates
- Secure authentication and authorization
- Efficient database management
- Supports future enhancements such as payment integration, QR code verification, notifications, and mobile applications.